

Iteracyjny projekt systemu symulacji programowania w Ladder

Wiktor Jankowski-Ostrowski

1 Wstęp

Dokument opisuje iteracyjny rozwój systemu sterowania w symulatorze PLC. Każda iteracja rozszerza funkcjonalność poprzedniej i wprowadza nowe możliwości z punktu widzenia implementacji, testowania oraz nauki podstaw sterowania. Całość została podzielona na cztery iteracje:

1. Bare minimum – absolutna podstawa, bez której dalsza praca nie jest możliwa
2. Iteracja wprowadzająca niewielkie usprawnienia, które realnie umożliwiają dalszą rozbudowę programu
3. Jedna z najważniejszych i najciekawszych iteracji w całym projekcie
4. Ostatnia iteracja, traktowana bardziej jako rozszerzenie i ciekawostka niż konieczność

Źródło ogólne do pogłębienia wiedzy:

PLC Programming – The Engineering Projects

2 Iteracja 1 – Bare Minimum

Pierwsza iteracja definiuje absolutne minimum funkcjonalne wymagane do stworzenia działającego programu sterowania. Jest to punkt wyjścia całego projektu – bez tej iteracji nie ma realnej możliwości tworzenia sensownych algorytmów sterowania.

2.1 Wejścia i wyjścia

W tej części definiowane są podstawowe mechanizmy wejść i wyjść cyfrowych, które stanowią fundament programowania w LD.

- Wejścia i wyjścia cyfrowe:
 - Normalnie otwarte (NO)
 - Normalnie zamknięte (NC)
 - Set
 - Reset
- Możliwość konfiguracji wyjść analogicznie jak wejść

Materiał uzupełniający:

NO i NC w Ladder Logic

2.2 Zmienne

Zmienne stanowią podstawowy sposób przechowywania stanu programu oraz wartości przetwarzanych w trakcie jego działania.

- Typ BOOL
- Typ INT
- Wartości typu INT mogą reprezentować sygnały analogowe

Materiał uzupełniający:
Typy danych w PLC

2.3 Liczniki

Liczniki umożliwiają zliczanie zdarzeń oraz realizację prostych algorytmów sekwencyjnych.

- Licznik zliczający w górę
- Licznik zliczający w dół
- Licznik dwukierunkowy

Materiał uzupełniający:
Liczniki w PLC

2.4 Timery

Timery pozwalają na wprowadzanie zależności czasowych w logice sterowania.

- Timer ON
- Timer OFF

Materiał uzupełniający:
Timery w PLC

2.5 Operacje logiczne i porównania

Operacje logiczne oraz komparatory umożliwiają podejmowanie decyzji w programie na podstawie wartości zmiennych.

- Komparatory: równe, większe, mniejsze
- Operatory logiczne: AND, OR, IF
- Detekcja zbocza: narastające i opadające

Materiał uzupełniający:
Komparatory logika w PLC

2.6 Podsumowanie iteracji

Po zakończeniu tej iteracji możliwa jest nauka podstaw programowania LD oraz tworzenie prostych algorytmów sterowania. Brakuje jeszcze wygody i modularności, jednak iteracja ta jest w pełni wystarczająca do nauki podstaw.

3 Iteracja 2 – Rozszerzenie podstaw

Druga iteracja rozszerza funkcjonalność Iteracji 1 bez jej przebudowy. Jej celem jest poprawa czytelności programu oraz przygotowanie gruntu pod dalszy rozwój.

3.1 Czytelność i organizacja

- Komentarze do rungów
- Komentarze do bloków funkcyjnych

Materiał uzupełniający:

Dokumentowanie programów PLC

3.2 Nowe elementy logiczne

- Zmienna typu rejestr blokowy
- Blok MOVE
- Licznik dwukierunkowy (UP/DOWN)

Materiał uzupełniający:

Instrukcja MOVE

3.3 Operacje na bitach

Operacje bitowe umożliwiają bezpośrednią manipulację danymi na poziomie bitów, co jest szczególnie przydatne przy pracy z rejestrami.

- Przesuwanie bitów w lewo
- Przesuwanie bitów w prawo

Materiał uzupełniający:

Operacje na bitach w PLC

3.4 Rozszerzenia funkcjonalne

Na tym etapie warto również wprowadzić elementy o charakterze bardziej edukacyjnym.

- Funkcje matematyczne: ABS, MOD, MAX, MIN
- Monitorowanie wartości zmiennych analogowych w czasie

Materiał uzupełniający:

Obliczenia matematyczne w Ladder Logic

3.5 Obserowanie zmiennych w czasie

Tutaj ten podpunkt ma dosyć dużą wartość edukacyjną całości może być tutaj generowanie od razu schematów logiki działania programów w postaci maszyny stanów jednak jest to tylko usapwienie. Najważniejszą cechą będzie po prostu możliwość obserwacji zmiennych wartości w czasie co pozwoli również na testowanie bardziej zaawansowanych algorytmów.

3.6 Podsumowanie iteracji

Na tym etapie możliwe jest już stworzenie rozbudowanych programów, włącznie z implementacją komunikacji pomiędzy urządzeniami. Brak Iteracji 3 powoduje jednak, że dalsza rozbudowa jest uciążliwa, choć nadal możliwa.

4 Iteracja 3 – Funkcje zaawansowane

Trzecia iteracja skupia się na rozszerzalności i ponownym wykorzystaniu logiki, co znacząco zwiększa wygodę pracy z projektem.

4.1 Bloki użytkownika

Jednym z kluczowych elementów tej iteracji jest możliwość tworzenia własnych bloków funkcyjnych na podstawie wcześniej napisanego programu w języku LD. Pozwala to na hermetyzację logiki oraz jej wielokrotne wykorzystanie.

Przykładowy program w Ladder Logic

Rozważmy prosty program składający się z dwóch wejść oraz jednego wyjścia.

- I_START – wejście cyfrowe (BOOL)
- I_ENABLE – wejście cyfrowe (BOOL)
- Q_OUT – wyjście cyfrowe (BOOL)

Logika programu:

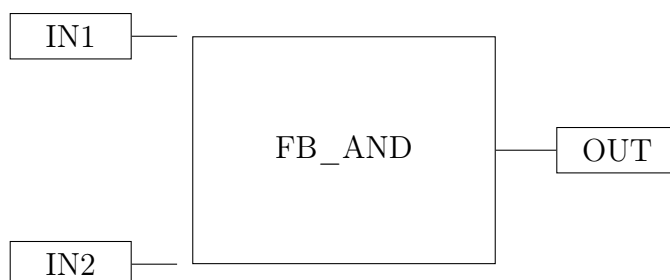
$$Q_OUT = I_START \wedge I_ENABLE$$



Powyższy rung realizuje funkcję logiczną AND. Wyjście zostaje aktywowane wyłącznie wtedy, gdy oba wejścia są w stanie wysokim.

Zapis programu jako blok funkcyjny

Ten sam fragment logiki zostaje zapisany jako blok funkcyjny. System automatycznie określa interfejs bloku na podstawie użytych sygnałów.



Opis działania bloku

- IN1 – odpowiada sygnałowi I_START
- IN2 – odpowiada sygnałowi I_ENABLE
- OUT – wyjście logiczne realizujące operację AND

Po utworzeniu bloku funkcyjnego może on zostać wielokrotnie użyty w programie głównym, bez konieczności ponownego tworzenia tej samej logiki w LD. Takie podejście znacząco poprawia czytelność programu i ułatwia dalszą rozbudowę.

4.2 Rozszerzenia

- Bloki czasowe użytkownika
- Dodatkowe typy timerów
- Kolejna zaawansowana funkcja matematyczna

Po tej iteracji system staje się w pełni funkcjonalnym i elastycznym środowiskiem do programowania LD, umożliwiającym budowę nawet złożonych algorytmów. Tutaj również jeżeli patrzymy to na bardziej strone jako Cisco to tutaj można już naprawdę łatwo to zrobić całości

5 Iteracja 4 – Integracja i dalszy rozwój

Ostatnia iteracja skupia się na regulatorach i dalszej rozbudowie systemu.

- Wprowadzenie regulatora PID wraz z możliwością podglądu i modyfikacji parametrów
- (Opcjonalnie) wprowadzenie innych regulatorów jako gotowych bloków

Materiał uzupełniający:
Regulator PID w PLC